



Monitores



Un monitor es una ABSTRACCIÓN que:

- **Encapsula un recurso crítico** (variables compartidas).
- Proporciona **procedimientos** para acceder a ese recurso.
- Garantiza **exclusión mutua**: solo un hilo puede estar ejecutando código del monitor a la vez.
- Permite **sincronización** mediante **variables de condición**.

Explicación

Para nosotros será otro mecanismo que tengamos para controlar la concurrencia, y de nuevo tendremos que diferenciar entre la teoría y la implementación.

Monitor teórico

Por su propia definición, un monitor garantiza la exclusión mutua, nos da igual como. Si algo está dentro del monitor, sabemos que estará en exclusión mutua.

A través de su uso centralizamos el acceso al recurso y los métodos para acceder a él.

En el modelo teórico clásico el monitor tiene:

- Una **cola de exclusión mutua** (hilos que quieren entrar).
- Una **cola de espera por cada variable de condición** (aquí sí podemos tener más de una variable de condición, no como en las regiones críticas condicionales).

A nivel teórico, la colas de un monitor sería 1 (exclusión mutua) + n (número de variables de condición).

```

monitor M:
  var recurso
  cond c

  procedure esperar():
    while not condicion do
      wait(c)
    end

  procedure usar():
    ... sección crítica ...
end monitor

```

Podemos simularla con un semáforo.

Simulación con semáforos

Para simular un monitor necesitas:

- 1 **semáforo binario** → exclusión mutua del monitor
- 1 **semáforo general por variable de condición**
- variables auxiliares para contar hilos esperando

```

semaphore mutex = 1    // exclusión mutua del monitor
semaphore colaC = 0    // cola de la condición
int esperandoC = 0    // número de procesos esperando en C
semaphore siguiente = 0 // mecanismo SU (signal and urgent)

procedure entrar_monitor():
  wait(mutex)

procedure salir_monitor():
  if esperandoC > 0 then
    signal(colaC)
  else

```

```

    signal(mutex)
end if

procedure waitC():
    esperandoC := esperandoC + 1
    signal(mutex)    // cedo el monitor
    wait(colaC)      // me bloqueo
    // cuando me despiertan (tengo el monitor)
    esperarC := esperarC - 1

procedure signalC():
    if esperandoC > 0 then
        signal(colaC)
    end if

```

Peculiaridades de la implementación

Hay algo **MUY IMPORTANTE** que debemos tener en cuenta cuando pasamos a implementar un monitor VS hablar de él en términos teóricos.

Es la razón de por qué se usa `while` en implementación, mientras que en la teoría las condición se evalúan con un `if`.

A nivel **teórico** (modelo de Hoare), cuando un proceso hace:

```
if not condición then wait(c)
```

y otro hace:

```
signal(c)
```

la semántica ideal es:

- quien hace `signal(c)` **cede inmediatamente** el monitor al proceso despertado,
- y se garantiza que **cuando el proceso despertado reanuda**, la condición lógica que estaba esperando **es verdadera**.

Por eso, en teoría, se suele escribir el esquema con `if`.

En lenguajes como **Java, C++ o Python** no podemos garantizar prioridades ni el orden exacto de reanudación.

El sistema **no asegura** que cuando un hilo despierta:

- la condición siga siendo verdadera,
- nadie haya modificado el estado compartido,
- o incluso que haya sido despertado por un `signal` (puede haber despertares espurios).

Por eso el esquema correcto es:

```
while not condición do
  wait(c)
end while
```

El `while` protege al monitor de todas estas irregularidades:

- si el hilo despierta "por error", vuelve a comprobar;
- si otro hilo cambia la condición antes de que reanudemos, volvemos a esperar;
- si se despiertan varios hilos y solo uno puede avanzar, los demás vuelven a bloquearse.

Porque en los lenguajes reales **no existe la semántica fuerte de Hoare** y no hay garantías sobre prioridades ni sobre el estado del recurso cuando el hilo se despierta.

Y ahora, implementemos el monitor.

Implementación

No tiene por qué llamarse "monitor" en el lenguaje.

Basta con tener:

- Exclusión mutua (cerrojos, `synchronized`, `mutex` ...)
- Sincronización (condiciones de guarda + algo tipo `wait/notify` o `condition_variable`).

Vamos, que si tenemos una clase en la que **TODOS LOS MÉTODOS** están protegidos en exclusión mutua, y nos ofrece sincronización, eso es un monitor.

Java API Standart

En la API Standart NO PODEMOS realizar una implementación directa de los monitores, ya que no tenemos variables de condición.

Sin embargo podemos simular su comportamiento utilizando `synchronized(obj)` y los métodos `wait()`, `notify()`, y `notifyAll()`.

En Java, todo objeto hereda de la clase `Object`.

Esta clase, por definición, tiene un cerrojo asociado y una cola de espera (wait-set). **SOLO UNA COLA DE ESPERA POR OBJETO**. No podemos emular múltiples variables de condición del modelo teórico.

Por esto podemos decir que todo objeto es "un monitor potencial".

No existe la semántica que propone Hoare ni prioridades. Todo monitor en Java usará la política de señalización de señalar y continuar.

```
public class Buffer {

    private int dato;
    private boolean lleno = false;

    public synchronized void producir(int x) throws InterruptedException {
        while (lleno) {
            wait(); // cola de condición (wait-set)
        }
        dato = x;
        lleno = true;
        notifyAll(); // despierta consumidores
    }

    public synchronized int consumir() throws InterruptedException {
        while (!lleno) {
            wait();
        }
    }
}
```

```

    lleno = false;
    notifyAll(); // despierta productores
    return dato;
}
}

```

Java API Alto Nivel

Gracias a la clase de `ReentrantLock` y las variables de condición `Condition` sí que podemos tener varias variables de condición como en el modelo teórico.

No obstante, al no poder garantizar prioridades nuestra semántica será siempre señalar y continuar, por lo que tendremos solo un COMPORTAMIENTO SIMULADO, no una IMPLEMENTACIÓN DIRECTA REAL.

```

class BoundedBuffer<T> {

    private final T[] items;
    private int count, in, out;

    private final Lock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    BoundedBuffer(int size) {
        items = (T[]) new Object[size];
    }

    public void put(T x) throws InterruptedException {
        lock.lock();
        try {
            while (count == items.length)
                notFull.await();

            items[in] = x;
            in = (in + 1) % items.length;

```

```

        count++;

        notEmpty.signal();
    } finally {
        lock.unlock();
    }
}
}

```

C++

Tampoco tenemos monitores nativos, pero sí que podemos simular su comportamiento.

Desde **C++11** tenemos:

- `std::mutex` , `std::unique_lock` → exclusión mutua.
- `std::condition_variable` → colas de espera.

Con ellos se pueden implementar **monitores reales**:

- Una clase que encapsula:
 - el estado compartido,
 - un `std::mutex` ,
 - una o varias `std::condition_variable` ,
 - y proporciona **métodos públicos** que actúan como procedimientos del monitor.

No es un monitor “nativo” del lenguaje, pero sí una **implementación que simula el comportamiento del modelo teórico**.

Seguimos sin poder tener una IMPLEMENTACIÓN DIRECTA porque C++ tampoco tiene prioridades, así que su política de señalización por defecto será señalar y continuar.

```

#include <mutex>
#include <condition_variable>

```

```

#include <vector>

class BoundedBuffer {
private:
    std::vector<int> buffer;
    std::mutex m;
    std::condition_variable no_lleno;
    std::condition_variable no_vacio;

    int in = 0;
    int out = 0;
    int count = 0;

public:
    explicit BoundedBuffer(size_t size)
        : buffer(size) {}

    void put(int x) {
        std::unique_lock<std::mutex> lk(m);

        while (count == (int)buffer.size()) {
            no_lleno.wait(lk);    // libera m y se bloquea
        }

        buffer[in] = x;
        in = (in + 1) % buffer.size();
        ++count;

        no_vacio.notify_one();    // despierta a un consumidor
    }

    int take() {
        std::unique_lock<std::mutex> lk(m);

        while (count == 0) {
            no_vacio.wait(lk);
        }
    }

```



```
}

int x = buffer[out];
out = (out + 1) % buffer.size();
--count;

no_lleno.notify_one(); // despierta a un productor
return x;
}
};
```